# VORTEXA 3.0 — Problem Statement Booklet

**12-hour student hackathon · AIRS, Dr. D. Y. Patil Institute of Technology, Pimpri, Pune**

## Contents


**Part 1 — Master Index** ..... **2**
**Part 2 — Problem Statements** ..... **4**
Domain 1 — Cybersecurity & Digital Trust ..... 4
C-01 — ActionGate: AI Agent Action Authorization Engine ..... 4
C-02 — BreachPath: Attack-Path Discovery & Remediation ..... 5
C-03 — PromptShield: Prompt Injection & Data-Exfiltration Defense ..... 6
Domain 2 — AI Systems & Infrastructure ..... 7
A-01 — ModelRouter: Cost-, Latency- & Quality-Aware AI Routing ..... 7
A-02 — GreenSched: Deadline-Aware Compute Job Scheduler ..... 8
A-03 — IdentityGuard: Session Risk & Account-Takeover Detection ..... 9
Domain 3 — Blockchain & Decentralized Trust ..... 10
B-01 — ProofChain: Verifiable Records & Tamper Detection ..... 10
B-02 — InsureChain: Policy-Aware Insurance Claims Workflow ..... 11
B-03 — MilestoneVault: Adversarial Smart-Contract Escrow ..... 12
Domain 4 — Supply Chain & Optimization ..... 13
S-01 — ReRoute: Dynamic Fleet & Delivery Replanning ..... 13
S-02 — StockSense: Adaptive Inventory Replenishment Engine ..... 14
S-03 — OrderWise: Priority-Based Order Allocation ..... 15
Domain 5 — AI-Powered Education & Learning Technology ..... 16
E-01 — LearnLoop AI: Adaptive AI Tutor & Misconception Detector ..... 16
E-02 — CodeQuest AI: AI Coding Mentor & Debugging Missions ..... 17
E-03 — TeachSense AI: AI Classroom Insight & Lesson Copilot ..... 18
Domain 6 — Public Systems & Civic Technology ..... 19
P-01 — RationSaathi: Public Distribution Entitlement & Dispute Assistant ..... 19
P-02 — CaseCompass: Legal Case Timeline & Action Organizer ..... 20
P-03 — RailFlow: Railway Disruption & Platform Conflict Simulator ..... 21
Domain 7 — Government Systems & GovTech ..... 22
G-01 — RozgarLink: Skills-to-Opportunity Matching Engine ..... 22
G-02 — GovFix: Configurable Public-Institution Workflow Engine ..... 23
G-03 — BudgetClear: Public Spending Reconciliation & Anomaly Review ..... 24

# Part 1 — Master Index

## Event Introduction

VORTEXA 3.0 is a 12-hour hackathon for undergraduate engineering students. Teams of two to four choose one of the 21 problem statements below and build a working software prototype for it. The booklet describes each problem and what must be addressed. It does not prescribe a solution: teams decide the approach, architecture, and tools.

## Common Rules

- **Deliverable:** a working software MVP that judges can test live. Mockups, static dashboards, slides, and conceptual designs alone are not enough.
- **Reproducibility:** the solution must support reproducible live testing, using inputs permitted by the problem statement.
- **AI:** every solution must use AI meaningfully, for a task that benefits from language understanding, classification, matching, forecasting, extraction, or adaptation. A chatbot or an LLM call that only rewrites text does not qualify. Teams choose their own AI approach and may combine it with deterministic logic, validation, and optimization.
- **Correctness:** where a decision has real consequences (eligibility, authorization, arithmetic, scheduling), the outcome must follow explicit, checkable rules. AI may inform such decisions but must not silently override them.
- **Data:** no organizer dataset is provided. Use manual inputs, your own files, team-generated scenarios, or small built-in examples. Use fictional data only; do not use real personal records, real financial transactions, or live third-party systems.
- **Infrastructure:** no paid API, paid cloud service, GPU, or special hardware is required. External AI services may be used, but the core demonstration must not depend on them; provide a local fallback.

## Domains and Problem Statements

| Domain | Code | Title |
|---|---|---|
| **1. Cybersecurity & Digital Trust** | C-01 | ActionGate: AI Agent Action Authorization Engine |
| | C-02 | BreachPath: Attack-Path Discovery & Remediation |
| | C-03 | PromptShield: Prompt Injection & Data-Exfiltration Defense |
| **2. AI Systems & Infrastructure** | A-01 | ModelRouter: Cost-, Latency- & Quality-Aware AI Routing |
| | A-02 | GreenSched: Deadline-Aware Compute Job Scheduler |
| | A-03 | IdentityGuard: Session Risk & Account-Takeover Detection |
| **3. Blockchain & Decentralized Trust** | B-01 | ProofChain: Verifiable Records & Tamper Detection |
| | B-02 | InsureChain: Policy-Aware Insurance Claims Workflow |
| | B-03 | MilestoneVault: Adversarial Smart-Contract Escrow |
| **4. Supply Chain & Optimization** | S-01 | ReRoute: Dynamic Fleet & Delivery Replanning |
| | S-02 | StockSense: Adaptive Inventory Replenishment Engine |
| | S-03 | OrderWise: Priority-Based Order Allocation |

| Domain | Code | Title |
|---|---|---|
| **5. AI-Powered Education & Learning Technology** | E-01 | LearnLoop AI: Adaptive AI Tutor & Misconception Detector |
| | E-02 | CodeQuest AI: AI Coding Mentor & Debugging Missions |
| | E-03 | TeachSense AI: AI Classroom Insight & Lesson Copilot |
| **6. Public Systems & Civic Technology** | P-01 | RationSaathi: Public Distribution Entitlement & Dispute Assistant |
| | P-02 | CaseCompass: Legal Case Timeline & Action Organizer |
| | P-03 | RailFlow: Railway Disruption & Platform Conflict Simulator |
| **7. Government Systems & GovTech** | G-01 | RozgarLink: Skills-to-Opportunity Matching Engine |
| | G-02 | GovFix: Configurable Public-Institution Workflow Engine |
| | G-03 | BudgetClear: Public Spending Reconciliation & Anomaly Review |

# Part 2 — Problem Statements

## Domain 1 — Cybersecurity & Digital Trust

## C-01 — ActionGate: AI Agent Action Authorization Engine

**Domain:** 1 — Cybersecurity & Digital Trust

### *Problem Overview*

AI agents now act rather than only answer: they delete records, send messages, update accounts, or trigger payments. Their interpretation of a request can be wrong, and text they read can be written to manipulate them. Many setups execute whatever action the model proposes, with at most a coarse list of allowed tools. Such checks ignore the specific arguments, the number of records affected, whether the action can be undone, and whether the approved version is the one that actually runs. When something goes wrong, it is also hard to reconstruct who or what decided to allow it.

### *The Challenge*

Build a gateway that every proposed agent action must pass through before it can run. Each action must end as allowed, blocked, or held for human approval. Unknown agents and unrecognized action types must not be allowed by default.

- Interpreting an action's intent and impact may inform the decision, but explicit rules about permissions, impact, reversibility, and policy must make the final call.
- An approval must apply only to the exact action approved. If any element changes afterwards, the action must be reauthorized before it can run. Requesters should not approve their own requests.
- Every decision, including refusals, must be recorded in a history that makes later tampering detectable.
- The system must keep making safe decisions when its AI component is unavailable or returns unusable output, and must record which path produced each decision.

## C-02 — BreachPath: Attack-Path Discovery & Remediation

**Domain:** 1 — Cybersecurity & Digital Trust

### *Problem Overview*

Access in an organization is a network of relationships. A user can assume a role, the role lets a service read a database, and that service is reachable from an internet-facing application. Each link may look reasonable on its own, but together they can form a route from an outside entry point to sensitive data. Reviews that check permissions one at a time miss these multi-step routes, and relationships change constantly, so a route closed last month may be open today. Analysing these combinations is difficult because the number of possible routes grows quickly and relationships can loop back on themselves.

### *The Challenge*

Build a tool that models users, services, roles, permissions, trust relationships, and sensitive assets, entered manually or uploaded as a small file. Users must be able to mark entry points and sensitive assets.

- Identify the routes from entry points to sensitive assets, analysing relationships across multiple steps, within a configurable maximum path length. Loops in the model must not cause failure or endless analysis.
- Present each route as an ordered sequence of steps with the relationship type at each step, explained in plain language.
- Suggest changes that would break a route. Suggestions must refer only to relationships that exist in the model, and the user decides whether to apply them.
- After any change, recalculate the routes from the updated model and show exactly which routes were removed, which remain, and which are new.
- Reject malformed models with a clear message. Do not invent missing elements or assume relationships the user did not enter.

## C-03 — PromptShield: Prompt Injection & Data-Exfiltration Defense

**Domain:** 1 — Cybersecurity & Digital Trust

### *Problem Overview*

Language-model applications mix trusted developer instructions with untrusted text: user messages, uploaded documents, web pages, and tool outputs. Malicious text can be written to look like an instruction. In a direct injection, a user tells the assistant to ignore its rules and reveal hidden instructions. In an indirect injection, the text is hidden in content the assistant retrieves, such as a product review, and the user never sees it. Either attack can aim to expose protected information or send data elsewhere. Keyword blocklists miss paraphrases and wrongly flag harmless requests, so context matters.

### *The Challenge*

Build a middleware layer that screens prompts and simulated tool outputs before they reach the language model. Treat content according to its source: text returned by a tool or retrieved from a document is less trusted than text typed by the user.

- Detect content that tries to change the assistant's instructions, reveal protected information, or send data to an outside destination. Detection should consider context, and should catch protected values even when spacing, capitalization, or simple encoding is changed.
- Give every input a clear outcome: allow, block, sanitize (remove only the offending part, preserving legitimate content), or escalate for review. Outcomes must follow an explicit, editable policy, with a recorded reason.
- Test both direct and indirect injection. Measure false positives as well as detections, using a labelled set of malicious and legitimate examples, and report misses honestly.
- Untrusted content must never be able to change the policy. Instructions found inside documents or tool outputs are data to examine, not commands to follow.

Domain 2 — AI Systems & Infrastructure

## A-01 — ModelRouter: Cost-, Latency- & Quality-Aware AI Routing

**Domain:** 2 — AI Systems & Infrastructure

### *Problem Overview*

Applications often have access to several language models that differ in capability, speed, price, context length, and reliability. A short factual question suits a small, cheap model; a long contract analysis may need a larger one. Sending every request to the strongest model wastes money, while sending everything to the cheapest model hurts quality on hard tasks. Endpoints also fail or slow down, and a feature hard-coded to one model simply breaks. The difficulty is that each request has several constraints at once (latency, cost, capability, context length), the best choice can change as conditions change, and a fallback chosen in a hurry can silently break a constraint the original choice respected.

### *The Challenge*

Build an inference gateway that chooses a model for each request. Each request carries explicit constraints: a maximum latency, a maximum cost, a minimum capability level, and where relevant a required context length. These are hard conditions; a model that violates any of them is not a valid choice.

- Among valid models, choose according to a stated preference, such as the lowest cost, and make every decision explainable: show which models were considered and why others were rejected.
- Treat endpoints as unreliable. If a chosen model fails or exceeds its latency budget, fail over to another model that still meets every constraint. Failover must never relax a constraint.
- If no model can meet the constraints, say so clearly rather than sending the request to an invalid model.
- Request complexity may have to be estimated from the content. Treat such estimates as uncertain, and when unsure, choose the safer option.
- Track cost, latency, failures, and routing decisions so that behaviour can be reviewed. Judges must be able to change a model's latency or availability and see a valid alternative selected.

## A-02 — GreenSched: Deadline-Aware Compute Job Scheduler

**Domain:** 2 — AI Systems & Infrastructure

### *Problem Overview*

Shared computing environments run training jobs, data pipelines, and reports on a limited number of machines. Each job has a resource need, a priority, and a deadline. Simple queues such as first-come-first-served can block an urgent job behind a long, low-priority one, and they ignore energy and cost. Job durations are usually guesses, and machines fail in the middle of the day, invalidating schedules that were correct a moment earlier. Scheduling is therefore not a single decision but a continuing response to change.

### *The Challenge*

Build a scheduler that assigns jobs to a limited pool of machines. Each job has a description, resource requirement, priority, deadline, and duration. Where a duration is unknown, estimate it as a range with a stated confidence, rather than presenting a guess as fact.

- The schedule must never exceed a machine's capacity at any time, and no job may run on a machine lacking the resources it needs. Meet deadlines first, then reduce estimated energy use and cost.
- Use a real scheduling or optimization method that you can explain, and state its limitations honestly; it need not find the best possible schedule.
- Document how energy and cost are estimated from stated machine assumptions, so a reviewer can reproduce them.
- Compare your approach against a simple baseline on the same workload, reporting deadline misses, utilization, and estimated energy and cost.
- When a machine fails or a priority changes, produce a valid revised schedule. Affected jobs must be rescheduled or reported as unscheduled with a reason.

## A-03 — IdentityGuard: Session Risk & Account-Takeover Detection

**Domain:** 2 — AI Systems & Infrastructure

### *Problem Overview*

Account takeover usually begins with someone logging in from a device and location the owner has never used, then performing sensitive actions such as changing an email address or requesting a transfer. Detecting this early limits the damage. Detection is hard because legitimate behaviour also looks unusual: travellers log in from new cities, people change phones, and some users rarely log in at all. Blanket rules create many false alarms, while a rule that checks only for known bad sources misses careful attackers. Alerts that say only “suspicious login” are also hard for analysts to act on.

### *The Challenge*

Build a session-monitoring engine that receives events containing a timestamp, user, device identifier, location label, and action. It should learn each user’s usual pattern from earlier events and use it to judge new activity.

- Detect and explain risk signals, such as an unfamiliar device, an implausible location change for the time elapsed, unusual activity hours for that user, and sensitive actions shortly after a login from an unfamiliar device. Show each signal’s value and reason.
- Handle sparse history, unknown location labels, and out-of-order timestamps, and state when a judgement rests on limited information.
- Apply a configurable response: no action, step-up verification, session revocation, or analyst review. Responses must follow explicit, readable rules so every outcome is traceable to its signals.
- AI may help recognise suspicious patterns in action sequences, but it must not trigger a response alone; explicit rules must also support it.
- Let an analyst replay stored events to see the signals and responses they would produce. A revoked session must be refused any further actions.

Domain 3 — Blockchain & Decentralized Trust

## B-01 — ProofChain: Verifiable Records & Tamper Detection

**Domain:** 3 — Blockchain & Decentralized Trust

### *Problem Overview*

Certificates, contracts, and official records are often stored by one custodian, and people must trust its database. If the same database holds a fingerprint of each record, whoever can alter the record can usually alter the fingerprint too, so the stored value proves nothing to an outside party. A verifier needs a reference that the custodian cannot quietly rewrite, must be able to check a single record without downloading the whole registry, and needs to know *why* a check failed. Verification proves that a document matches a registered reference; it does not prove who created the document or that its contents are true.

### *The Challenge*

Build a document registry where records are fingerprinted with the SHA-256 hash function, computed from a defined, reproducible form of each document. Records are grouped into batches summarized by a Merkle tree, with each batch's root published. Every record must be able to produce a membership proof for its batch.

- Published roots must be stored where the registry cannot silently rewrite them, so later changes are detectable.
- An independent verifier must confirm or reject a record using only the document, its proof, and the published root, without access to the registry database. It must report which check failed, such as a modified document, an invalid proof, or a root mismatch.
- Detect a modified document, an invalid proof, and a stored record that no longer matches its published reference. The verifier must still reject a tampered document even if the registry database has been altered.
- AI may help extract metadata (issuer, date, document type), suggest categories, and explain failures in plain language. Extracted data must be confirmed by a person, and AI must not affect whether verification succeeds.
- No public blockchain is required. Verification must be genuine cryptographic checking, not a visual indicator.

## B-02 — InsureChain: Policy-Aware Insurance Claims Workflow

**Domain:** 3 — Blockchain & Decentralized Trust

### *Problem Overview*

An insurance claim is a request for payment under a policy, and deciding it means applying several rules together: coverage limits, exclusions, and a deadline for filing after an incident. Adjusters gather evidence and record decisions; supervisors approve larger amounts; and the same incident must not be paid twice. Manual handling makes errors easy: a late claim is approved, a duplicate slips through under a slightly different reference, or an approval is recorded by someone without authority. Narratives can also contradict structured fields, which deserves a human look but is not proof of wrongdoing.

### *The Challenge*

Build a claims workflow using only simulated policies and claims. Each policy defines a coverage limit, excluded categories, and a filing window. Each claim includes its policy, incident and filing dates, category, amount, narrative, and evidence references.

- Decide eligibility with explicit rules and report the rule that determined each outcome, including claims filed late, in excluded categories, or above the limit. Define and consistently apply a rule for claims above the limit.
- Detect duplicates, meaning claims about the same incident under the same policy even with different references.
- Claims move through defined states such as submitted, under review, approved, rejected, and paid. Only permitted transitions are allowed, only authorized roles may make them, and skipping a state is refused.
- Keep a history of every accepted decision and every refusal, protected against undetected edits.
- AI may extract key fields from the narrative and flag inconsistencies with the structured data. These flags are prompts for review only; they must not change eligibility or state, and must not be presented as proof of fraud.

## B-03 — MilestoneVault: Adversarial Smart-Contract Escrow

**Domain:** 3 — Blockchain & Decentralized Trust

### *Problem Overview*

Milestone-based agreements are common in freelance work, construction, and grant funding: payment follows approved deliverables, and disputes are common. Escrow is usually held by an intermediary whose rules are not visible. A contract can make the conditions explicit, but code that controls money is unforgiving. A payment released twice cannot easily be recovered, a missing access check lets the wrong person take funds, and a re-entrant call can drain funds before state is updated. Agreement text, meanwhile, is written in prose with ambiguous conditions that must be resolved before any money is committed.

### *The Challenge*

Build a milestone-based escrow in which simulated funds are released or refunded only when the agreement's conditions are met. The agreement names a client, a provider, and an arbiter who resolves disputes, and defines each milestone's amount and approval conditions.

- Implement the core logic as a Solidity smart contract, tested on a local Hardhat network or an equivalent local setup. No real cryptocurrency and no public deployment are required.
- Enforce the lifecycle: the provider marks a milestone complete; the client approves it (releasing funds) or disputes it (passing the decision to the arbiter); the arbiter releases or refunds disputed funds. Each step is limited to the party allowed to take it, and undefined transitions are rejected.
- Prevent duplicate releases, unauthorized refunds, and out-of-order transitions. The contract's balance must always reconcile with deposits and payments.
- Back each guarantee with automated tests covering normal and adversarial actions, including a second release attempt, an unauthorized refund, an out-of-order transition, and a re-entrancy attack.
- AI may convert agreement text into a proposed checklist of milestones, amounts, conditions, and ambiguities. A person must confirm it, ambiguities must be resolved, and AI output must never be written to the contract directly.

Domain 4 — Supply Chain & Optimization

## S-01 — ReRoute: Dynamic Fleet & Delivery Replanning

**Domain:** 4 — Supply Chain & Optimization

### *Problem Overview*

Each morning, dispatchers decide which vehicle carries which deliveries and in what order. Vehicles have capacities, deliveries have sizes and time windows, and travel times vary. The plan rarely survives the day: a bridge closes, a vehicle breaks down, or an urgent delivery arrives. Dispatchers must then decide what to move and what to tell customers, often with a spreadsheet and experience. Changing the plan quickly is hard, and an unchecked change can overload a vehicle, miss a window, or undo a completed stop.

### *The Challenge*

Build a delivery-planning engine that assigns deliveries to vehicles while respecting capacity and time windows. Inputs include the depot, vehicles, deliveries, and a matrix of simulated travel times; no live mapping or GPS service may be used.

- Implement a routing or allocation algorithm that you can explain, and state its limits. Show arrival times at each stop, including any waiting before a window opens.
- Never overload a vehicle at any point on its route. A delivery that cannot be placed must be reported with the reason.
- Support three disruptions: a road closure that increases travel time between two locations for a stated period, a vehicle failure at a stated time, and an urgent delivery added during the day. Disruptions may be structured or described in text.
- Where a text report is interpreted, the dispatcher must confirm the resulting changes before they take effect. A report naming an unknown location must be rejected, not guessed at.
- Replanning starts from the current state: completed stops stay fixed, and a failed vehicle receives no stops after its failure time.
- Show the original and revised plans side by side, classifying every delivery as kept, moved, or broken, with a reason, so the dispatcher can explain changes to customers.

## S-02 — StockSense: Adaptive Inventory Replenishment Engine

**Domain:** 4 — Supply Chain & Optimization

### *Problem Overview*

Every stocked product raises the same question: when to reorder, and how much. Ordering late causes stockouts; ordering early ties up money and storage, and risks waste for perishable goods. The right decision depends on demand and its variation, supplier lead time and its reliability, and the cost of ordering and holding stock. Many operations set a reorder level once and never revisit it, so a promotion-driven surge or a late supplier causes stockouts that the data could have predicted. Sales history also contains unusual values, such as one-off spikes or data-entry errors, which distort any forecast if not handled carefully.

### *The Challenge*

Build a replenishment tool that recommends when and how much to reorder for each product, using demand history, supplier lead time and its variability, ordering cost, holding cost, and unit cost.

- Implement real inventory calculations, including a reorder point and safety stock that protect against variation in demand and lead time. State the formulas and assumptions used.
- The target service level is a parameter selected by the user. Every recommendation must display the service level and the statistical factor it uses.
- Identify unusual demand values, keep them visible, and classify them (for example, promotion, data error, or unknown) so they are handled differently. AI may help classify them and propose a forecast adjustment, but any adjustment must stay within a defined range and be shown.
- Compare the forecast with a simple baseline on a holdout period the forecast did not use, so any added sophistication is measured rather than assumed.
- Let users simulate a demand surge or a supplier delay, and show the revised recommendation, expected cost, and stockout risk beside the previous one.

## S-03 — OrderWise: Priority-Based Order Allocation

**Domain:** 4 — Supply Chain & Optimization

### *Problem Overview*

When stock cannot fill every order, someone must decide which orders to fill. The decision depends on contract priorities, penalty clauses, promised dates, minimum acceptable quantities, and whether partial delivery is allowed. Manual decisions are inconsistent, spreadsheets can promise the same unit twice, and customers who receive a partial or deferred order need a clear reason. A decision that helps one urgent order can silently displace several routine orders, and this effect is easy to miss.

### *The Challenge*

Build an allocation engine that distributes limited stock among competing orders according to explicit priorities and delivery commitments. Each order has a product, requested quantity, minimum acceptable quantity, partial-delivery permission, base priority, and optional promised date.

- Every allocation must be valid: it must never exceed available stock, never give an order less than its minimum, and never partially fill an order that does not accept partial delivery.
- Allocation must follow a documented, reviewable rule based on priority and promised date.
- Business priorities are often written as free-text notes. AI may read such notes and propose a priority weight with a reason, but a person must confirm it. Notes must not be able to override stock levels, minimums, or delivery rules.
- Each order ends as fulfilled, partially fulfilled, or deferred, with a reason a coordinator could give the customer.
- When stock or priorities change, recalculate and show every order whose quantity or status changed. An urgent order that exceeds remaining stock is the central case to handle.

## Domain 5 — AI-Powered Education & Learning Technology

# E-01 — LearnLoop AI: Adaptive AI Tutor & Misconception Detector

**Domain:** 5 — AI-Powered Education & Learning Technology

### *Problem Overview*

A wrong answer is often a symptom. A student who says 2/5 is larger than 2/3 "because five is bigger" is applying a whole-number rule to fractions, while another student who makes the same error may be confused about something else entirely. Generic feedback and static quizzes treat them the same, and chatbot tutors answer questions without tracking what a learner has actually understood. A single good score can also reflect memory of one question rather than mastery of an idea, so improvement is hard to measure.

### *The Challenge*

Build an interactive tutor for one topic, using material the team prepares or the participants provide, broken into concepts with questions and explanations.

- Explain each concept clearly, using examples, analogies, and step-by-step reasoning. Explanations must be accurate; generated wording must stay faithful to verified material.
- Ask follow-up questions and analyse each answer. Identify the likely misconception behind an incorrect answer, distinguish it from a random slip, and ask a clarifying question when the answer is too unclear to diagnose.
- Two learners who give different wrong answers to the same question must receive different diagnoses and different follow-up activities.
- Track mastery for each concept based on demonstrated performance over time. The record must reflect evidence from the learner's work, not a typed claim.
- Support a repeatable pre- and post-assessment using the same items, and report the change along with the sample size and its limits.
- The tutor supports learning; it must not present itself as a replacement for a teacher.

## E-02 — CodeQuest AI: AI Coding Mentor & Debugging Missions

**Domain:** 5 — AI-Powered Education & Learning Technology

### *Problem Overview*

Beginners learn programming by writing code and reading what goes wrong, but feedback is often hard to use. A traceback names an error without explaining it, and a program can run and still give the wrong answer. Learners often copy complete solutions, which gets code working but skips the debugging that builds skill. Practice tools that only report pass or fail do not show where to look, and tools that give full answers too early remove the struggle that makes learning stick. Running learner code also carries risk: code from an unknown author must not read sensitive files, reach the network, or consume unlimited resources.

### *The Challenge*

Build a practice environment for one programming language, with challenges at increasing difficulty. Each challenge has a description, starter code, visible tests, and hidden tests. Choose a language your team can support well within the build time.

- Execute learner submissions only in a restricted environment. Full sandboxing is not required; if it is not feasible, a constrained local test runner is acceptable, but unrestricted execution of learner code is not.
- Report each test's outcome separately: passed, wrong output, runtime error, or timeout. Never reveal the content of hidden tests.
- When a submission fails, explain the failure in plain language to help the learner find the problem.
- Provide progressive hints. Each hint must be more specific than the last, but no hint may reveal the complete solution, including AI-generated hints.
- Detect recurring mistakes across a learner's work, such as off-by-one errors or unhandled empty inputs, and adapt future challenges to them. Track demonstrated skills by concept.

## E-03 — TeachSense AI: AI Classroom Insight & Lesson Copilot

**Domain:** 5 — AI-Powered Education & Learning Technology

### *Problem Overview*

After an assessment, a teacher must decide what to teach next and to whom. Question-level scores show where a class struggled but not why: two students who both answered incorrectly may hold entirely different misconceptions, and a lesson aimed at one may do nothing for the other. Writing separate explanations and practice for each group takes time teachers rarely have. Follow-up assessments are also seldom compared with the first round in a structured way, so it is hard to tell whether a particular intervention worked.

### *The Challenge*

Build a teacher-facing tool that takes assessment questions, answer keys, and student responses, entered manually or uploaded. Each response must link to a known student and question; unmatched responses must be reported, not discarded.

- Identify the likely misconception behind each incorrect response, using a label set the teacher defines or confirms. Show confidence, and flag uncertain classifications for teacher review.
- Group students by demonstrated learning need using a rule the teacher can understand and check. Handle awkward cases explicitly: students with several misconceptions, students with no clear pattern, and ties.
- For each group, propose targeted material: an explanation, a revision activity, and follow-up questions. These are drafts; they must not be visible to students until the teacher approves them.
- After a follow-up assessment, compare results overall, by group, and for the questions targeting each group's misconception. State the number of students, and do not claim an intervention worked when the evidence is too thin.
- Use fictional student data only. Include a demonstration class with at least two distinct misconceptions.

## Domain 6 — Public Systems & Civic Technology

# P-01 — RationSaathi: Public Distribution Entitlement & Dispute Assistant

**Domain:** 6 — Public Systems & Civic Technology

### *Problem Overview*

Public distribution schemes provide subsidised goods to eligible households at set quantities for set periods. Collections may be partial or spread over several visits, receipts may be lost or hard to read, and beneficiaries often have no simple way to compare what they are owed with what they received. A clear, evidence-based comparison can resolve simple recording errors and make a grievance easier to review. The calculation itself is simple, but its inputs are messy, and a discrepancy may have benign causes, such as genuine stock shortages. A shortfall is not evidence of wrongdoing, and presenting it as one would cause real harm.

### *The Challenge*

Build a tool that compares a beneficiary's expected entitlement with the quantities actually received, and prepares a structured discrepancy report.

- Entitlements must be expressed through explicit rules the user enters, such as a quantity per member per period. Apply only the rules provided; do not assume others.
- Record distributions, including partial deliveries spread over a period, and calculate shortfalls with exact arithmetic that the user can check line by line.
- Link every record to its source, such as a note, receipt reference, or pasted receipt text, and keep that evidence in the report.
- Receipt text may be interpreted to extract quantities and dates, and a discrepancy may be classified (for example, shortfall, late, unrecorded, or unclear). Extracted values are suggestions the user must confirm, and arithmetic must not depend on AI.
- The report must use neutral language, describe the difference and its evidence, and leave the cause open. It must never state or imply that anyone has committed fraud.

## P-02 — CaseCompass: Legal Case Timeline & Action Organizer

**Domain:** 6 — Public Systems & Civic Technology

### *Problem Overview*

A legal case produces many documents over time: court orders, filings, notices, hearing dates, and deadlines, often written in unfamiliar formal language. Someone supporting a party must know what happened, in what order, and what must be done by when, and a missed deadline can have serious consequences. Records are fragile: dates may conflict across documents, an order may be referred to but not included, and deadlines may be buried in the text. A timeline built from such material can easily fill gaps with plausible guesses, and an invented deadline is as harmful as a missed one.

### *The Challenge*

Build a tool that organizes supplied case events and orders into a chronological timeline, linking every entry to the document it came from. Use only fictional or supplied case material; live court systems must not be used or assumed.

- Parse dates and place events in order. A date that cannot be read with confidence must be flagged, not guessed.
- When two records give different dates for the same event, show both with their sources and flag the conflict.
- Identify gaps: a referenced order that is missing, an event without a date, or a sequence that seems to skip a step.
- Extract only deadlines the supplied records explicitly state, each linked to its source sentence. Never calculate a deadline unless the user supplies the rule, and never invent one.
- AI may extract events and explain terminology supplied by the user. Every AI-extracted item must point to its source text; items without a source are rejected.
- The tool must not predict legal outcomes and should make clear that its output organizes the record and is not legal advice.

## P-03 — RailFlow: Railway Disruption & Platform Conflict Simulator

**Domain:** 6 — Public Systems & Civic Technology

### *Problem Overview*

A station has a limited number of platforms, and each train needs one for its arrival, its departure, and the time in between. A platform cannot host two trains whose occupancy overlaps, and a minimum gap is usually needed between occupancies. Timetables respect these limits only until something changes: a train runs late, a platform closes after an incident, and a delay can spread to other services using the same platform. Operators must find a feasible revision quickly, understand the trade-offs, and be sure the revised timetable contains no conflict that a manual check might miss.

### *The Challenge*

Build a simulator for a small fictional station, defined by its platforms and the minimum gap required between occupancies. A timetable lists trains, arrival and departure times, and any platform a train must use. No live railway data may be used.

- Assign trains to platforms so that no platform is double-occupied and every required gap is respected. Detect overlapping or infeasible assignments and report each conflict by train and platform.
- Support delays of stated length and platform closures for stated periods. After a disruption, replan the affected trains, respecting the same constraints and recording the reasons for each change.
- Validate every revised timetable with a check that is independent of the process that produced it, confirming there are no overlaps or violated gaps. A timetable that fails validation must not be presented as valid.
- AI may convert a free-text disruption description into specific delays and closures, and may explain trade-offs between revisions in plain language. AI must not decide platform assignments; those must follow from the constraints.

## Domain 7 — Government Systems & GovTech

# G-01 — RozgarLink: Skills-to-Opportunity Matching Engine

**Domain:** 7 — Government Systems & GovTech

### *Problem Overview*

Job seekers and trainees struggle to find suitable opportunities. Listings describe the same skill in different words, and a qualified candidate may never see a role because a keyword search did not match. Two different things are often mixed together: eligibility conditions, such as a required qualification or residence, which a candidate must meet; and preferences, such as commute or pay, which only affect ranking. Treating a requirement as a preference recommends jobs a candidate cannot formally take, and treating a preference as a requirement hides options a candidate could legitimately pursue.

### *The Challenge*

Build a matching system that connects candidate profiles with manually entered opportunities. Profiles record skills, qualifications, location, and preferences; opportunities record required skills, eligibility criteria, location, and the work involved.

- Apply mandatory eligibility criteria separately from preference-based ranking. An opportunity whose eligibility conditions a candidate does not meet must not appear as an eligible match, however well the skills fit. Rank only eligible opportunities.
- Recognise skills described in different words as the same skill. AI may extract skills and judge how closely they correspond, and these judgements may inform ranking. They must never decide eligibility, which follows the criteria stated by the opportunity.
- Explain every result in plain language: why an eligible match ranks where it does, and which specific condition a candidate fails.
- Identify skill gaps for opportunities a candidate is interested in and suggest learning pathways from a catalogue the team supplies. Do not assume live access to any course provider.
- Changes to a candidate's qualifications or location must update eligibility and rankings immediately and visibly. The results are decision support; the criteria used are those stated by the opportunity.

## G-02 — GovFix: Configurable Public-Institution Workflow Engine

**Domain:** 7 — Government Systems & GovTech

### *Problem Overview*

Public institutions receive constant requests: complaints about municipal services, university service requests, hospital outpatient queues, electricity-bill disputes, and passenger assistance. Each request must be received, understood, assigned to the right person, acted on, and closed, and escalated if it misses its committed time. When this runs on email and spreadsheets, requests stall, deadlines pass unnoticed, and nobody can say who changed a status or when. Citizens hear nothing for weeks, officers cannot see which cases are overdue, and institutions cannot demonstrate fair treatment.

### *The Challenge*

Implement one complete workflow from these five tracks: municipal grievance resolution, university service requests, hospital OPD queue management, electricity-bill disputes, or passenger assistance. Each team selects one track and implements it end to end. A generic dashboard covering all five does not meet the challenge.

- Define the states a request can occupy and the transitions allowed between them. Refuse any skipped state or invalid move, with a clear reason.
- Restrict each status change to the roles permitted to make it, such as citizen, officer, or supervisor. Enforce these roles in the system itself, not only in the interface, and record refused attempts.
- Give each request a deadline appropriate to its track. Identify overdue requests and escalate them according to the workflow's rules, visibly and on record.
- Keep an audit history of accepted changes, refused attempts, assignments, and escalations, showing who did what and when.
- AI may categorise an incoming request, extract key details, or draft a suggested response. These are recommendations shown alongside the request. They must not change its status, assignment, or deadline; only the workflow and authorized people may do that.

## G-03 — BudgetClear: Public Spending Reconciliation & Anomaly Review

**Domain:** 7 — Government Systems & GovTech

### *Problem Overview*

Public budgets are recorded as large collections of expenditure lines with dates, descriptions, categories, amounts, and references, often exported from spreadsheets and combined across offices. Totals should equal the sum of their lines, and each category total should match its lines. In practice, the same payment may appear twice, a line may lack a category, and a stated total may not match its items. Finding these by reading rows is slow and error-prone. Reviewers also notice unusual entries, which deserve a closer look but are not proof of wrongdoing. Confusing a statistical oddity with a verified error can cause real harm.

### *The Challenge*

Build a budget-review tool that reconciles expenditure records supplied as CSV files or manual entries. Records include date, description, category, amount, and reference, and the tool must handle fields that are missing or unclear.

- Reconcile figures exactly: check that line items add up to stated totals and that category amounts match stated category totals. Report each verifiable discrepancy with the lines involved and the size of the difference.
- Detect duplicate entries, missing fields, and inconsistent figures. Show a missing category as missing rather than filling it in silently.
- AI may propose categories for ambiguous descriptions with a confidence level, and may suggest unusual entries for review with a short reason. These suggestions must stay separate from reconciliation and must never change a total or a verified discrepancy.
- The report must give an evidence-linked explanation for each flag, clearly separating verified arithmetic discrepancies from AI-generated suspicions.
- State that an anomaly flag is not proof of corruption or wrongdoing and that every flag needs human investigation before any conclusion is drawn.